AI Project Report: Lunar Lander Agent

Deep Q-Learning with Experience Replay & Polyak Target Network Updates

Group: 2205 | Environment: LunarLander-v3 | Algorithm: DQN (PyTorch) | Mean Reward: +281.82 (Solved ≥ 200)

1. Introduction & Task Formulation

The **LunarLander-v3** environment models a challenging 2D rocket lander physics simulation where an autonomous agent must navigate from random initial attitudes to a safe landing on a landing pad flanked by two flags.

State Space $S \subset \mathbb{R}^8$

1-2. Position coordinates (x, y)
3-4. Linear velocities (x_dot, y_dot)
5-6. Angle theta and angular velocity theta_dot
7-8. Ground contact flags for left and right landing legs

Action Space $A = \{0, 1, 2, 3\}$

0: Do nothing (inertial drift)
1: Fire left orientation engine
2: Fire main descent engine
3: Fire right orientation engine

Reward Structure: Rewards scale with proximity to the landing pad (+100 to +140), leg contact (+10 each), and +100 bonus for successful touchdown, counterbalanced by penalties for fuel expenditure (-0.3 main engine, -0.03 side engines) and crash failure (-100). The environment benchmark requires an average score of ≥ +200 over 100 consecutive episodes.

2. Theoretical Framework & Methodology

We formulate the problem as a Markov Decision Process (MDP) defined by $\langle S, A, P, R, \gamma \rangle$. The objective is finding a policy π that maximizes the expected discounted cumulative return:

$$J(\pi) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \quad \text{with discount factor } \gamma = 0.995$$

2.1 Q-Learning Objective & Bellman Optimality

The optimal action-value function $Q^*(s, a)$ represents the expected return from state s taking action a and thereafter acting optimally. Q^* satisfies the Bellman Optimality Equation:

$$Q^*(s, a) = E\left[R(s, a) + \gamma \max_{a' \in A} Q^*(s', a') \mid s, a \right]$$

2.2 Deep Neural Network Parameterization

Due to the continuous 8-dimensional observation space, tabular representations are infeasible. We approximate $Q^*(s, a)$ via a deep neural network $Q(s, a; \theta)$. The architecture comprises a 4-layer Multi-Layer Perceptron (MLP) with non-linear ReLU activations:

$$\mathbb{R}^8 \to \text{Linear}(128) \to \text{ReLU} \to \text{Linear}(128) \to \text{ReLU} \to \text{Linear}(64) \to \text{ReLU} \to \text{Linear}(4) \to \mathbb{R}^4$$

2.3 Loss Function & Mean Squared Bellman Error

The Q-network parameters θ are trained by minimizing the Mean Squared Bellman Error (MSBE) across sampled minibatches:

$$L(\theta) = E_{(s, a, r, s', d) \sim D}\left[\left(r + \gamma (1 - d) \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

where $d \in \{0, 1\}$ denotes episode termination, and θ^- denotes the parameters of the decoupled target network.

2.4 Experience Replay Buffer

Sequential interaction transitions $(s_t, a_t, r_t, s_{t+1}, d_t)$ suffer from severe autocorrelation, violating i.i.d. optimization assumptions. We utilize an experience replay buffer D of capacity $N = 100{,}000$, drawing uniformly distributed batches of size $B = 256$ for updates.

2.5 Exploration vs. Exploitation (ε-Greedy Strategy)

Action selection incorporates exploration during initial learning and exploitation as the policy matures:

$$a = \text{random_action} \in A \quad \text{with prob } \varepsilon; \qquad a = \arg\max_{a'} Q(s, a'; \theta) \quad \text{with prob } (1 - \varepsilon)$$

ε initializes at 1.0 and decays geometrically: $\varepsilon_{k+1} = \max(0.05, \varepsilon_k \times 0.999)$.

2.6 Polyak Target Network Soft Updates

To prevent harmful policy oscillations caused by a rapidly shifting optimization target, a separate target network $Q(s, a; \theta^-)$ is updated smoothly every 4 steps via Polyak averaging:

$$\theta^- \leftarrow \tau\, \theta + (1 - \tau)\, \theta^- \quad \text{with tracking coefficient } \tau = 0.05$$

3. Implementation & Optimization Architecture

The complete machine learning pipeline is developed in Python utilizing **PyTorch** for autograd calculations and **Gymnasium** for environment simulation.

Parameter Serialization

In accordance with the `best_policy_2205.npy` specification, PyTorch tensors are converted into a zero-dimensional NumPy structured array with exact dtypes, ensuring portable, lightweight storage (**105 KB**).

Inference Optimization & Caching

In `policy_2205.py`, we implement lazy network caching to reconstruct the model once per evaluation lifecycle rather than on every frame, reducing total 100-episode runtime to **~31.5 seconds**.

4. Empirical Results & Quantitative Benchmark

The trained policy was evaluated using the official course benchmark harness (`evaluate_agent.py`) across 100 test episodes (5 rendered in human mode, 95 in rgb_array mode):

+281.82	100%	31.55s	105 KB
MEAN REWARD (100 EPS)	SAFE LANDING RATE	EXECUTION TIME (< 100S)	WEIGHT FILE SIZE (< 1 MB)

Metric / Parameter	Configured / Observed Value	Specification Target	Compliance Status
Evaluation Mean Reward	**+281.82 ± 18.4**	≥ +200.0 (Environment Solved)	**✓ Surpassed (+81.82)**
Evaluation Runtime (100 eps)	**31.55 seconds**	≤ 100.0 seconds	**✓ Compliant (68% margin)**
Policy Parameter File Size	**105,040 bytes (~105 KB)**	≤ 1,048,576 bytes (1 MB)	**✓ Compliant (10% of limit)**
Discount Factor (γ)	0.995	Hyperparameter	Preserves long-term horizon
Optimizer & Learning Rate (α)	Adam, $\alpha = 5 \times 10^{-4}$	Hyperparameter	Stable gradient propagation
Replay Buffer & Minibatch Size	N = 100,000, B = 256	Hyperparameter	Breaks temporal correlations
Polyak Soft Update Rate (τ)	τ = 0.05, freq = 4 steps	Hyperparameter	Guarantees stable Bellman targets

5. Conclusion & Viva Voce Defense Summary

The Deep Q-Network successfully solves LunarLander-v3 with near-optimal fuel conservation and upright landing stabilization. In the viva voce examination, our approach is justified as follows:

- **Convergence Stability:** The combination of soft Polyak target updates (τ = 0.05) and decoupled replay memory prevents the classic "moving target" divergence phenomenon in off-policy Q-learning.
- **Sample Efficiency:** Storing 100,000 transitions ensures rare, high-reward touchdown experiences are repeatedly replayed, accelerating policy convergence.
- **Operational Robustness:** The decaying ε-greedy policy ensures full state-space exploration early in training, avoiding local minima such as premature engine cutoff or ground avoidance.

6. Project Team Details (Group 2205)

Per submission guidelines, the student with the minimum roll number serves as group leader responsible for submitting all deliverables.

Roll Number	Student Name	Department	Role
2401137	**OM RAI**	**B.Tech Artificial Intelligence**	**Group Leader**
2401158	RAMADUGULA LAKSHMI SURYA KARTHIKEYA SAI PRANAV	B.Tech Artificial Intelligence	Team Member
2401179	SAYAN SHIT	B.Tech Artificial Intelligence	Team Member
2401203	SOMYA PATEL	B.Tech Artificial Intelligence	Team Member
2401225	VANSH PRATAP SINGH SYLONIA	B.Tech Artificial Intelligence	Team Member